Sistemas de Tiempo Real — Práctica 3

Ejercicio 5: Planificación FIFO

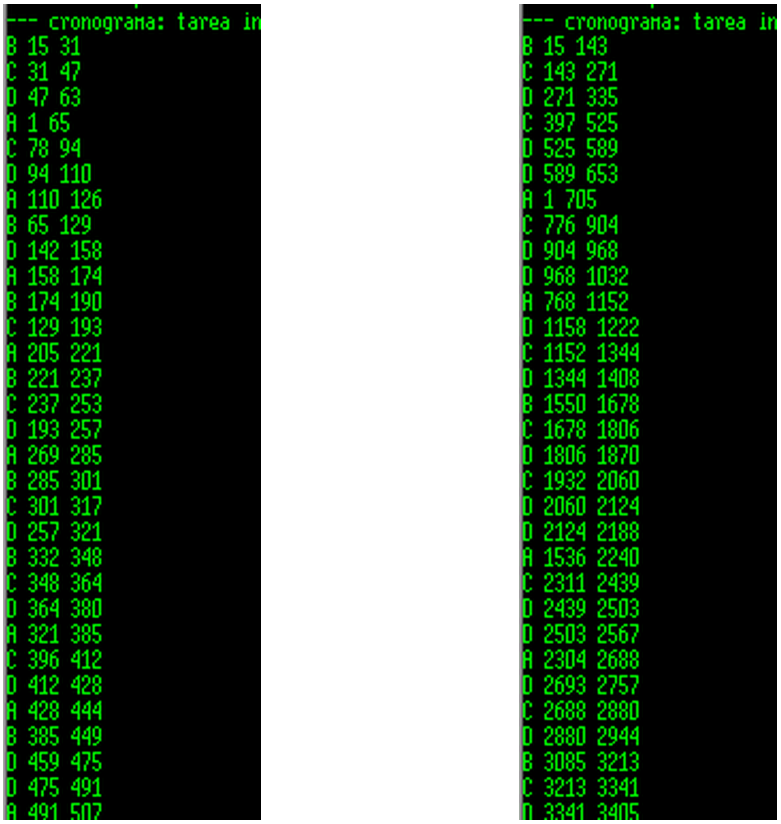
Ingeniería en Computación

Método

Se mantienen las cuatro tareas del Ejercicio 1 con igual prioridad y se deshabilita el reparto por cuantos. Según indica el enunciado, en `FreeRTOSConfig.h` de la biblioteca `configUSE_TIME_SLICING` pasa de 1 a 0 y `configUSE_PREEMPTION` queda habilitado. Además, por lo explicado en el informe del Ejercicio 4, en este banco el cuanto no lo produce la interrupción del tick sino la propia tarea con `taskYIELD()` al cerrar cada tramo de 16 ms, de modo que pasar a FIFO requiere también dejar de ceder. Son dos valores, uno en la configuración de la biblioteca y otro en el firmware, y el proyecto no compila si quedan desacoplados.

Una observación previa que conviene declarar. En el Ejercicio 4 se hizo la prueba de control inversa: dejar `configUSE_TIME_SLICING` en 1 y quitar únicamente el `taskYIELD()`. El resultado fue *idéntico* al de este ejercicio, hasta el milisegundo. Es decir que en este simulador el parámetro de la biblioteca no tiene efecto observable, porque la interrupción del tick no llega durante el cómputo y el núcleo nunca rota entre tareas de igual prioridad. Lo que produce el comportamiento FIFO que se mide acá es, en los hechos, la ausencia de `taskYIELD()`.

Para comparar los cronogramas sin depender de la configuración del analizador lógico, el firmware registra en memoria los tramos de ejecución (cuándo cada tarea toma y suelta el procesador) y los imprime por la terminal con los instantes en milisegundos.



Round Robin FIFO

Figura 1: Cronogramas de ejecución, en milisegundos: tarea, instante de inicio e instante de fin de cada tramo.

a) Comparación cualitativa de los cronogramas

La diferencia es inmediata. Bajo Round Robin los tramos duran **16 ms** y las cuatro tareas se turnan sin parar: `B 15 31`, `C 31 47`, `D 47 63`, `C 78 94`, `D 94 110`, `A 110 126`. En el mismo intervalo hay más de treinta cambios de contexto.

Bajo FIFO los tramos duran lo que dura el cómputo entero: `B 15 143` son 128 ms, `C 143 271` otros 128 y `D 271 335` son 64. En el mismo intervalo hay seis tramos en lugar de treinta.

Round Robin reparte el procesador en el tiempo; FIFO lo entrega de a uno y completo.

b) Atomicidad y expropiación

Las ráfagas son atómicas. Se verifica restando las dos columnas del cronograma: todos los tramos de A, B y C miden **128 ms** y los de D miden **64 ms**, que son exactamente sus tiempos de cómputo ($C = 2$ y $C = 1$ unidades,

con 1 unidad = 64 ms). Ninguna ráfaga aparece partida.

¿Sigue siendo expropiativo? Sí. Entre prioridades distintas la expropiación es la del sistema operativo: la tarea de fondo, que corre en la tarea Idle, es desalojada en cuanto se libera cualquier periódica. Lo que desaparece no es la expropiación sino el reparto por cuantos entre tareas de igual prioridad.

¿En qué situación podría producirse igualmente una expropiación? El cronograma la muestra, y la muestra en una sola tarea. A nunca llega a ejecutar sus 128 ms de corrido:

Activación de A	Tramos de ejecución (ms)	Suma	Resto
1.ª	`1-15`, `335-396`, `652-705`	128	atómicas
2.ª	`768-776`, `1032-1152`	128	atómicas
3.ª	`1536-1549`, `1869-1931`, `2187-2240`	128	atómicas

A C le ocurre de vez en cuando: `C 1152 1344` aparece partida en `1152-1157` y `1221-1344`. B y D nunca se parten.

La causa está en los bordes donde se reanuda el planificador. El despachador de activaciones llama a `vTaskSuspendAll()` y `xTaskResumeAll()`, y al reanudar FreeRTOS procesa los ticks que quedaron pendientes durante la suspensión; si alguno llega a procesarse y hay otra tarea lista, `xTaskResumeAll()` cede el procesador antes de devolver el control. Por eso los cortes caen siempre en múltiplos de los 16 ms con que el firmware trocea la ráfaga para llamar al despachador, y por eso son esporádicos: dependen de que justo en esa ventana haya un tick pendiente.

La expropiación ocurre entonces **en los bordes donde se reanuda el planificador**, aunque la tarea nunca ceda explícitamente. Son unos pocos cortes por hiperperíodo, muy lejos de constituir un reparto por cuantos: ese es exactamente el motivo por el cual el Ejercicio 4 necesita forzar el cuanto desde la tarea.

c) Orden de atención de tareas activadas en el mismo instante

Las cuatro tareas se crean en el orden A, B, C, D y el despachador las libera en ese mismo orden, con el planificador suspendido, de modo que quedan listas en el mismo instante. El orden observado de atención es, sin embargo,

$$\mathbf{B} \rightarrow \mathbf{C} \rightarrow \mathbf{D} \rightarrow \mathbf{A}$$

B toma el procesador a los 15 ms, C a los 143, D a los 271, y A recién completa a los 705. **No coincide con el orden de llegada.**

La explicación está en cómo FreeRTOS maneja la lista de listos. `prvAddTaskToReadyList` llama a `vListInsertEnd`, que inserta el elemento *delante del índice actual* de una lista circular; y la selección, `listGET_OWNER_OF_NEXT_ENTRY`, **avanza ese índice** antes de devolver el dueño. El índice no se reinicia entre selecciones: es estado que persiste de la vuelta anterior.

En consecuencia el recorrido es cíclico sobre la lista y arranca desde donde había quedado, no desde el primero que llegó. La lista de listos de FreeRTOS no es una cola FIFO por orden de llegada, aunque la política de ejecución sí sea no expropiativa. Es una distinción que conviene no confundir: lo *FIFO* del Ejercicio 5 es que la ráfaga no se interrumpe, no que se atienda en orden de llegada.

d) Resultados comparados

	Respuesta en el instante crítico		Plazos perdidos			
Tarea	Round Robin	FIFO	Round Robin	FIFO	T (ms)	
A	507 ms	**705 ms**	0	0	768	empeora
B	523 ms	**143 ms**	0	0	1536	mejora
C	529 ms	**271 ms**	1,00 por hiperper.	**0**	384	mejora
D	257 ms	**335 ms**	1,13 por hiperper.	**2,99 por hiperper.**	192	empeora

Mejoran B y C, que son las atendidas primero: B pasa de 523 a 143 ms, una reducción del 73 %, y C de 529 a 271. C además deja de perder plazos por completo.

Empeoran A y D. A pasa de 507 a 705 ms porque queda última en el recorrido de la lista y la pasan por encima las tareas que se reactivan mientras espera. D pasa de 257 a 335 ms y, sobre todo, multiplica por 2,6 sus plazos perdidos: de 1,13 por hiperperíodo a 2,99, porque con ráfagas atómicas puede quedar bloqueada hasta 128 ms detrás de una tarea cuyo plazo no corre riesgo.

El trabajo total no cambió y el procesador no queda ocioso ni un milisegundo en ninguna de las dos corridas mientras hay ráfagas pendientes: las cuatro tareas piden $3 \times 128 + 64 = 448$ ms de cómputo y el sistema los sirve igual de rápido en los dos casos. Lo que cambia es el *orden* en que los sirve, y por lo tanto a quién se le cobra

la espera. Bajo FIFO la última en ser atendida, A, termina a los 705 ms en lugar de 507, y la diferencia son exactamente las dos reactivaciones de D y la segunda ráfaga de C que se colaron adelante mientras A esperaba.

e) Conclusión

Ninguna de las dos políticas garantiza el cumplimiento de plazos. Round Robin reparte el perjuicio entre dos tareas, C y D, que pierden uno y 1,13 plazos por hiperperíodo respectivamente. FIFO lo concentra en una sola: C queda en cero pero D pasa a perder el 37,3 % de sus activaciones (103 plazos sobre 276 activaciones).

La información que ambos planificadores ignoran es la misma: **el período y el plazo de cada tarea**. Round Robin solo conoce el cuanto; FIFO solo conoce el orden de la lista de listos. Ninguno de los dos recibe el dato de que D tiene que terminar cada 192 ms mientras B dispone de 1536, de modo que tratan igual a tareas con urgencias que difieren en un factor de ocho.

De ahí que la respuesta no esté en elegir mejor entre estas dos, sino en cambiar la clase de política: asignar prioridades en función del período, como hace Rate Monotonic, o en función del plazo más próximo, como hace Earliest Deadline First. Con el conjunto de este trabajo, $U = 0{,}9167 < 1$, una planificación por prioridades según el período cumpliría todos los plazos sobre el mismo hardware y el mismo código de las tareas; lo único que cambia es qué sabe el planificador.